

Day 3 Misspecification Analysis

STAT778 Project

2026-04-24

Contents

Purpose	2
Background: what the pipeline is doing	2
Generation stage	2
Fitting stage	3
Load packages and project code	3
Load all Day 3 fit results	4
Inspect one fit-result object	4
Helper functions	5
Build the master fit table	7
Overview of the loaded Day 3 cells	9
Convergence summary	9
Runtime summary	10
Error messages for failed fits	11
Extract parameter estimates into long format	11
What does the truth mean here?	12
Which parameter names are present?	12
Estimate summary by cell, method, and parameter	13
MAE summary	14
Information criteria summary	16
Boxplots of runtime	16
Boxplots of parameter estimates	17
A simple 2 x 2 Day 3 summary table	18

Interpretation guide	19
1. Well-specified versus misspecified	20
2. GHK versus GQT under misspecification	20
3. Shared-parameter interpretation	20
Final takeaway	20

Purpose

This document analyzes the **Day 3 misspecification study**.

The current Day 3 design is a (2×2) matrix:

Generating family	Fitting family	Meaning
Poisson	Poisson	well-specified
Poisson	NegBin	misspecified
NegBin	NegBin	well-specified
NegBin	Poisson	misspecified

The goals of this analysis are:

1. verify that the misspecification runs completed successfully,
2. summarize convergence and runtime,
3. extract fitted parameter estimates,
4. compare estimates to the correct truth where that comparison is well-defined,
5. produce the first Day 3 tables and plots for the report.

Background: what the pipeline is doing

From the code standpoint, the Day 3 pipeline has two stages.

Generation stage

The generation script creates **scenario objects**. For the current misspecification pilot, there are two generating scenarios:

- `pois_pilot`
- `nbin_pilot`

Each scenario contains:

- `locations`: spatial coordinates
- `values`: a matrix of simulated datasets
- `config`: the full generating specification
- `true_params`: the flat generation truth

In both cases:

- the layout is a regular (7×7) grid, so $(n = 49)$,
- the number of Monte Carlo replicates is $(B = 30)$,
- the dependence structure is Matérn with:
 - `range` ($= 0.3$),

- ($\kappa = 0.5$),
- nugget (= 0).

Because ($\kappa = 0.5$), the Matérn reduces to the exponential correlation model

$$\rho(h) = \exp(-h/r),$$

with ($r = 0.3$).

The marginal families differ:

- `pois_pilot`: Poisson with mean ($\lambda = 5$),
- `nbin_pilot`: negative binomial with constant mean (5) and overdispersion `od = 1`.

So each replicate is a length-49 vector of spatially correlated counts.

Fitting stage

The fitting script loads a generated scenario and then fits each replicate with:

- GHK
- GQT

For Day 3, the fitting wrapper can override the marginal family used in fitting. That is how misspecification is implemented.

The key point is:

- the **generating family** comes from `scenario$config$marginal`,
- the **fitting family** comes from `fit_result$fit_marginal_spec`.

The fitting wrapper returns a **fit-result object** with one row per (`replicate`, `method`) pair.

So if one scenario has:

- 30 replicates
- 2 methods

then the resulting `fit_result$results` table has:

$$30 \times 2 = 60$$

rows.

Load packages and project code

```
suppressPackageStartupMessages({
  library(gcKrig)
})

source("code/R/logging.R")
source("code/R/io.R")
source("code/R/fit_core.R")
```

Load all Day 3 fit results

The Day 3 fitting script is expected to write fit-result objects under:

data/fits/MISSPEC/

We load every .rds file from that directory.

```
fit_dir <- file.path("data", "fits", "MISSPEC")
fit_files <- list.files(fit_dir, pattern = "\\..rds$", full.names = TRUE)

fit_files

## [1] "data/fits/MISSPEC/nbin_pilot__misfit_poisson.rds"
## [2] "data/fits/MISSPEC/nbin_pilot.rds"
## [3] "data/fits/MISSPEC/pois_pilot__misfit_negbin.rds"
## [4] "data/fits/MISSPEC/pois_pilot.rds"

if (length(fit_files) == 0L) {
  stop("No Day 3 fit-result files found under data/fits/MISSPEC/.")
}

fit_objects <- lapply(fit_files, readRDS)
names(fit_objects) <- basename(fit_files)

names(fit_objects)

## [1] "nbin_pilot__misfit_poisson.rds" "nbin_pilot.rds"
## [3] "pois_pilot__misfit_negbin.rds" "pois_pilot.rds"
```

Inspect one fit-result object

This helps remind us of the structure before building summaries.

```
example_name <- names(fit_objects)[1]
example_fit <- fit_objects[[1]]

cat("Example file:", example_name, "\n\n")

## Example file: nbin_pilot__misfit_poisson.rds
str(example_fit, max.level = 1)

## List of 9
## $ dataset_id      : chr "MISSPEC"
## $ scenario_id     : chr "nbin_pilot"
## $ scenario_config :List of 8
## $ fit_marginal_spec:List of 2
## $ misspecified    : logi TRUE
## $ results         : 'data.frame': 60 obs. of 11 variables:
## $ created_at      : POSIXct[1:1], format: "2026-04-22 12:32:33"
## $ r_version       : chr "R version 4.3.1 (2023-06-16)"
## $ gcKrig_version  : chr "1.1.8"
```

Helper functions

These helpers flatten the nested fit-result objects into analysis-ready tables.

```
safe_first <- function(x, default = NA) {
  if (length(x) == 0L || is.null(x)) return(default)
  x[[1]]
}

pretty_cell_label <- function(gen_family, fit_family) {
  paste0(gen_family, " \u2192 ", fit_family)
}

flatten_fit_result <- function(fr, source_file = NA_character_) {
  res <- fr$results

  out <- data.frame(
    dataset_id      = fr$dataset_id,
    gen_scenario_id = fr$scenario_id,
    source_file     = source_file,
    gen_family      = fr$scenario_config$marginal$family,
    fit_family      = fr$fit_marginal_spec$family,
    misspecified    = fr$misspecified,
    replicate       = res$replicate,
    method          = res$method,
    converged       = res$converged,
    error_message   = ifelse(is.na(res$error_message), "", res$error_message),
    runtime_sec     = res$runtime_sec,
    loglik          = res$loglik,
    aic             = res$aic,
    bic             = res$bic,
    aicc            = res$aicc,
    stringsAsFactors = FALSE
  )

  out$cell_label <- pretty_cell_label(out$gen_family, out$fit_family)

  # Preserve list-columns
  out$estimates <- res$estimates
  out$std_errors <- res$std_errors

  out
}

normalize_param_name <- function(pname) {
  p <- tolower(pname)
  p <- gsub("[[:space:]]+", "", p)
  p <- gsub("[`()]", "", p)
  p
}

truth_for_parameter <- function(param_name, gen_config, fit_marginal_spec) {
  p <- normalize_param_name(param_name)
```

```

gen_marg <- gen_config$marginal
gen_corr <- gen_config$corr
gen_family <- gen_marg$family
fit_family <- fit_marginal_spec$family

# range truth
if (grepl("range", p)) {
  return(gen_corr$range)
}

# kappa truth, only if ever estimated
if (grepl("kappa", p)) {
  return(gen_corr$kappa)
}

# nugget truth, only if ever estimated
if (grepl("nugget", p)) {
  return(gen_corr$nugget)
}

# overdispersion / od truth
if (grepl("^od$|overdisp|dispersion|disp", p)) {
  if (gen_family == "negbin" && fit_family == "negbin") {
    return(gen_marg$od)
  } else {
    return(NA_real_)
  }
}

# intercept / beta0 / mean-scale regression coefficient
if (grepl("intercept", p) || grepl("^beta0$", p) || grepl("^beta1$", p) ||
  grepl("^beta$", p) || grepl("^mu$", p)) {
  if (gen_family == "poisson") {
    return(log(gen_marg$lambda))
  }
  if (gen_family == "negbin") {
    return(gen_marg$beta[1])
  }
}

# fallback: if the generating model has only one marginal coefficient
# and the parameter isn't a known corr/dispersion parameter, treat it
# as the intercept-scale mean parameter
if (gen_family == "poisson") {
  return(log(gen_marg$lambda))
}
if (gen_family == "negbin") {
  return(gen_marg$beta[1])
}

NA_real_
}

```

```

extract_estimates_long <- function(flat_df, fit_objects_named) {
  rows <- list()
  idx <- 0L

  for (i in seq_len(nrow(flat_df))) {
    if (!isTRUE(flat_df$converged[i])) next

    est <- flat_df$estimates[[i]]
    if (is.null(est) || length(est) == 0L) next

    src_name <- flat_df$source_file[i]
    fr <- fit_objects_named[[src_name]]
    gen_config <- fr$scenario_config
    fit_spec <- fr$fit_marginal_spec

    for (pname in names(est)) {
      idx <- idx + 1L
      truth_val <- truth_for_parameter(pname, gen_config, fit_spec)

      rows[[idx]] <- data.frame(
        dataset_id      = flat_df$dataset_id[i],
        gen_scenario_id = flat_df$gen_scenario_id[i],
        source_file     = flat_df$source_file[i],
        gen_family      = flat_df$gen_family[i],
        fit_family      = flat_df$fit_family[i],
        misspecified    = flat_df$misspecified[i],
        cell_label      = flat_df$cell_label[i],
        replicate       = flat_df$replicate[i],
        method          = flat_df$method[i],
        parameter       = pname,
        estimate        = as.numeric(est[[pname]]),
        truth           = truth_val,
        abs_error       = if (is.na(truth_val)) NA_real_ else abs(as.numeric(est[[pname]]) - truth_val)
        stringsAsFactors = FALSE
      )
    }
  }

  if (length(rows) == 0L) {
    return(data.frame())
  }

  do.call(rbind, rows)
}

```

Build the master fit table

```

fit_df_list <- Map(
  function(obj, nm) flatten_fit_result(obj, source_file = nm),
  fit_objects,
  names(fit_objects)
)

```

```

)

fit_df <- do.call(rbind, fit_df_list)

dim(fit_df)

## [1] 240 18

head(fit_df)

##               dataset_id gen_scenario_id
## nbin_pilot__misfit_poisson.rds.1  MISSPEC      nbin_pilot
## nbin_pilot__misfit_poisson.rds.2  MISSPEC      nbin_pilot
## nbin_pilot__misfit_poisson.rds.3  MISSPEC      nbin_pilot
## nbin_pilot__misfit_poisson.rds.4  MISSPEC      nbin_pilot
## nbin_pilot__misfit_poisson.rds.5  MISSPEC      nbin_pilot
## nbin_pilot__misfit_poisson.rds.6  MISSPEC      nbin_pilot
##               source_file gen_family
## nbin_pilot__misfit_poisson.rds.1 nbin_pilot__misfit_poisson.rds      negbin
## nbin_pilot__misfit_poisson.rds.2 nbin_pilot__misfit_poisson.rds      negbin
## nbin_pilot__misfit_poisson.rds.3 nbin_pilot__misfit_poisson.rds      negbin
## nbin_pilot__misfit_poisson.rds.4 nbin_pilot__misfit_poisson.rds      negbin
## nbin_pilot__misfit_poisson.rds.5 nbin_pilot__misfit_poisson.rds      negbin
## nbin_pilot__misfit_poisson.rds.6 nbin_pilot__misfit_poisson.rds      negbin
##               fit_family misspecified replicate method
## nbin_pilot__misfit_poisson.rds.1      poisson      TRUE      1      GHK
## nbin_pilot__misfit_poisson.rds.2      poisson      TRUE      1      GQT
## nbin_pilot__misfit_poisson.rds.3      poisson      TRUE      2      GHK
## nbin_pilot__misfit_poisson.rds.4      poisson      TRUE      2      GQT
## nbin_pilot__misfit_poisson.rds.5      poisson      TRUE      3      GHK
## nbin_pilot__misfit_poisson.rds.6      poisson      TRUE      3      GQT
##               converged error_message runtime_sec      loglik
## nbin_pilot__misfit_poisson.rds.1      TRUE      0.83642387 -99.83277
## nbin_pilot__misfit_poisson.rds.2      TRUE      0.11296272 -103.06533
## nbin_pilot__misfit_poisson.rds.3      TRUE      1.14766049 -123.33175
## nbin_pilot__misfit_poisson.rds.4      TRUE      0.04071069 -123.33175
## nbin_pilot__misfit_poisson.rds.5      TRUE      0.85611129 -138.18027
## nbin_pilot__misfit_poisson.rds.6      TRUE      0.04992342 -151.28193
##               aic      bic      aicc      cell_label
## nbin_pilot__misfit_poisson.rds.1 203.6655 207.4492 203.9264 negbin → poisson
## nbin_pilot__misfit_poisson.rds.2 210.1307 213.9143 210.3915 negbin → poisson
## nbin_pilot__misfit_poisson.rds.3 250.6635 254.4472 250.9244 negbin → poisson
## nbin_pilot__misfit_poisson.rds.4 250.6635 254.4471 250.9244 negbin → poisson
## nbin_pilot__misfit_poisson.rds.5 280.3605 284.1442 280.6214 negbin → poisson
## nbin_pilot__misfit_poisson.rds.6 306.5639 310.3475 306.8247 negbin → poisson
##               estimates      std_errors
## nbin_pilot__misfit_poisson.rds.1 0.6863404, 0.1805235 0.21839532, 0.03619224
## nbin_pilot__misfit_poisson.rds.2 0.7705537, 0.1602491 0.17816112, 0.02813094
## nbin_pilot__misfit_poisson.rds.3 0.8266784, 0.0000000 NULL
## nbin_pilot__misfit_poisson.rds.4 0.8266784, 0.0000000 NULL
## nbin_pilot__misfit_poisson.rds.5 1.5581441, 0.1240376 0.11850167, 0.01986329
## nbin_pilot__misfit_poisson.rds.6 1.406497, 0.0000000 NULL

```


Overview of the loaded Day 3 cells

```
cell_overview <- unique(
  fit_df[, c("gen_scenario_id", "gen_family", "fit_family", "misspecified", "cell_label", "source_file")]
)

rownames(cell_overview) <- NULL
print(cell_overview)
```

```
##   gen_scenario_id gen_family fit_family misspecified      cell_label
## 1      nbin_pilot      negbin   poisson          TRUE negbin → poisson
## 2      nbin_pilot      negbin   negbin          FALSE negbin → negbin
## 3      pois_pilot      poisson   negbin          TRUE  poisson → negbin
## 4      pois_pilot      poisson   poisson         FALSE poisson → poisson
##                                source_file
## 1 nbin_pilot__misfit_poisson.rds
## 2                   nbin_pilot.rds
## 3 pois_pilot__misfit_negbin.rds
## 4                   pois_pilot.rds
```

This should show the four expected cells:

- Poisson (→) Poisson
- Poisson (→) NegBin
- NegBin (→) NegBin
- NegBin (→) Poisson

Convergence summary

We summarize convergence separately by:

- generating family,
- fitting family,
- estimation method.

```
convergence_tab <- aggregate(
  converged ~ gen_family + fit_family + cell_label + method,
  data = fit_df,
  FUN = function(x) c(
    n_total = length(x),
    n_ok     = sum(x),
    rate     = mean(x)
  )
)

convergence_summary <- data.frame(
  gen_family = convergence_tab$gen_family,
  fit_family = convergence_tab$fit_family,
  cell_label = convergence_tab$cell_label,
  method     = convergence_tab$method,
  n_total    = convergence_tab$converged[, "n_total"],
  n_ok       = convergence_tab$converged[, "n_ok"],
  conv_rate  = convergence_tab$converged[, "rate"]
)
```

```
print(convergence_summary, digits = 3)
```

##	gen_family	fit_family	cell_label	method	n_total	n_ok	conv_rate
## 1	negbin	negbin	negbin → negbin	GHK	30	30	1
## 2	negbin	poisson	negbin → poisson	GHK	30	30	1
## 3	poisson	negbin	poisson → negbin	GHK	30	30	1
## 4	poisson	poisson	poisson → poisson	GHK	30	30	1
## 5	negbin	negbin	negbin → negbin	GQT	30	30	1
## 6	negbin	poisson	negbin → poisson	GQT	30	30	1
## 7	poisson	negbin	poisson → negbin	GQT	30	30	1
## 8	poisson	poisson	poisson → poisson	GQT	30	30	1

Interpretation:

- A low convergence rate means that fitting is unstable for that cell.
- If misspecified cells converge much less often, that is already a meaningful result.

Runtime summary

We now summarize runtime among the converged fits.

```
runtime_df <- fit_df[fit_df$converged, ]
```

```
runtime_tab <- aggregate(
  runtime_sec ~ gen_family + fit_family + cell_label + method,
  data = runtime_df,
  FUN = function(x) c(
    mean = mean(x),
    median = stats::median(x),
    min = min(x),
    max = max(x),
    total = sum(x)
  )
)
```

```
runtime_summary <- data.frame(
  gen_family = runtime_tab$gen_family,
  fit_family = runtime_tab$fit_family,
  cell_label = runtime_tab$cell_label,
  method = runtime_tab$method,
  mean_sec = runtime_tab$runtime_sec[, "mean"],
  median_sec = runtime_tab$runtime_sec[, "median"],
  min_sec = runtime_tab$runtime_sec[, "min"],
  max_sec = runtime_tab$runtime_sec[, "max"],
  total_sec = runtime_tab$runtime_sec[, "total"]
)
```

```
print(runtime_summary, digits = 3)
```

##	gen_family	fit_family	cell_label	method	mean_sec	median_sec	min_sec
## 1	negbin	negbin	negbin → negbin	GHK	1.3448	1.2691	0.7012
## 2	negbin	poisson	negbin → poisson	GHK	0.9330	0.8497	0.4763
## 3	poisson	negbin	poisson → negbin	GHK	1.3319	1.2065	0.9075

```
## 4    poisson    poisson poisson → poisson    GHK    0.6581    0.6361 0.4898
## 5      negbin      negbin  negbin → negbin    GQT    0.2015    0.2051 0.1427
## 6      negbin    poisson  negbin → poisson    GQT    0.0925    0.1076 0.0387
## 7      poisson      negbin  poisson → negbin    GQT    0.1782    0.1766 0.1314
## 8      poisson    poisson poisson → poisson    GQT    0.0879    0.0855 0.0778
##    max_sec total_sec
## 1     2.389     40.35
## 2     2.957     27.99
## 3     3.441     39.96
## 4     1.244     19.74
## 5     0.253      6.05
## 6     0.207      2.77
## 7     0.270      5.34
## 8     0.151      2.64
```

Interpretation:

- GHK is expected to be slower than GQT.
- Misspecification may also change runtime if the likelihood becomes harder to optimize.

Error messages for failed fits

If any fits failed, inspect the error messages.

```
failed_df <- fit_df[!fit_df$converged, c(
  "gen_family", "fit_family", "cell_label", "replicate", "method", "error_message"
)]

if (nrow(failed_df) == 0L) {
  cat("No failed fits.\n")
} else {
  print(failed_df)
}
```

```
## No failed fits.
```

Extract parameter estimates into long format

The fitted estimates are stored in the list-column `fit_df$estimates`. We unpack them into one row per:

- generating family,
- fitting family,
- replicate,
- method,
- parameter.

We also attach a `truth` column where the comparison is well-defined.

```
est_long <- extract_estimates_long(fit_df, fit_objects_named = fit_objects)

dim(est_long)
```

```
## [1] 600 13
```

```
head(est_long)
```

```
##   dataset_id gen_scenario_id          source_file gen_family
## 1   MISSPEC      nbm_pilot nbm_pilot__misfit_poisson.rds  negbin
## 2   MISSPEC      nbm_pilot nbm_pilot__misfit_poisson.rds  negbin
## 3   MISSPEC      nbm_pilot nbm_pilot__misfit_poisson.rds  negbin
## 4   MISSPEC      nbm_pilot nbm_pilot__misfit_poisson.rds  negbin
## 5   MISSPEC      nbm_pilot nbm_pilot__misfit_poisson.rds  negbin
## 6   MISSPEC      nbm_pilot nbm_pilot__misfit_poisson.rds  negbin
##   fit_family misspecified      cell_label replicate method parameter estimate
## 1   poisson          TRUE negbin → poisson          1   GHK Intercept 0.6863404
## 2   poisson          TRUE negbin → poisson          1   GHK   range 0.1805235
## 3   poisson          TRUE negbin → poisson          1   GQT Intercept 0.7705537
## 4   poisson          TRUE negbin → poisson          1   GQT   range 0.1602491
## 5   poisson          TRUE negbin → poisson          2   GHK Intercept 0.8266784
## 6   poisson          TRUE negbin → poisson          2   GHK   range 0.0000000
##      truth abs_error
## 1 1.609438 0.9230975
## 2 0.300000 0.1194765
## 3 1.609438 0.8388842
## 4 0.300000 0.1397509
## 5 1.609438 0.7827595
## 6 0.300000 0.3000000
```

What does the truth mean here?

A key coding and interpretation point:

- For **Poisson-generated** data, the generation truth stores `lambda = 5`, but the fitted Poisson intercept is on the **log scale**, so the correct fitting-scale truth is:

$$\log(5) \approx 1.609.$$

- For **NegBin-generated** data, the current config uses:

$$\beta = (\log(5), 0, 0),$$

so the correct intercept-scale truth is again:

$$\log(5) \approx 1.609.$$

- The range truth is always (0.3).
 - The overdispersion truth is only well-defined for the **NegBin-generated**, **NegBin-fitted** cell. For misspecified fits, some parameters do not have a meaningful “true” comparison, so their truth is set to NA.
-

Which parameter names are present?

Before summarizing estimates, it helps to see what parameter names `gcKrig` actually returned.

```
sort(unique(est_long$parameter))
```

```
## [1] "Intercept"      "overdispersion" "range"
```

Estimate summary by cell, method, and parameter

```
estimate_tab <- aggregate(
  estimate ~ gen_family + fit_family + cell_label + method + parameter,
  data = est_long,
  FUN = function(x) c(
    mean = mean(x),
    sd = sd(x),
    median = stats::median(x),
    n = length(x)
  )
)

estimate_summary <- data.frame(
  gen_family = estimate_tab$gen_family,
  fit_family = estimate_tab$fit_family,
  cell_label = estimate_tab$cell_label,
  method = estimate_tab$method,
  parameter = estimate_tab$parameter,
  mean_est = estimate_tab$estimate[, "mean"],
  sd_est = estimate_tab$estimate[, "sd"],
  median_est = estimate_tab$estimate[, "median"],
  n = estimate_tab$estimate[, "n"]
)

print(estimate_summary, digits = 3)
```

##	gen_family	fit_family	cell_label	method	parameter	mean_est
## 1	negbin	negbin	negbin → negbin	GHK	Intercept	1.5420
## 2	negbin	poisson	negbin → poisson	GHK	Intercept	1.6031
## 3	poisson	negbin	poisson → negbin	GHK	Intercept	1.6316
## 4	poisson	poisson	poisson → poisson	GHK	Intercept	1.6348
## 5	negbin	negbin	negbin → negbin	GQT	Intercept	1.6011
## 6	negbin	poisson	negbin → poisson	GQT	Intercept	1.6270
## 7	poisson	negbin	poisson → negbin	GQT	Intercept	1.6353
## 8	poisson	poisson	poisson → poisson	GQT	Intercept	1.6349
## 9	negbin	negbin	negbin → negbin	GHK	overdispersion	0.9103
## 10	poisson	negbin	poisson → negbin	GHK	overdispersion	0.0360
## 11	negbin	negbin	negbin → negbin	GQT	overdispersion	1.1144
## 12	poisson	negbin	poisson → negbin	GQT	overdispersion	0.0431
## 13	negbin	negbin	negbin → negbin	GHK	range	0.2620
## 14	negbin	poisson	negbin → poisson	GHK	range	0.0901
## 15	poisson	negbin	poisson → negbin	GHK	range	0.3497
## 16	poisson	poisson	poisson → poisson	GHK	range	0.3073
## 17	negbin	negbin	negbin → negbin	GQT	range	0.3445
## 18	negbin	poisson	negbin → poisson	GQT	range	0.0925
## 19	poisson	negbin	poisson → negbin	GQT	range	0.3592

```
## 20    poisson    poisson poisson → poisson    GQT        range    0.3092
##      sd_est median_est  n
## 1  0.4789      1.513 30
## 2  0.5316      1.545 30
## 3  0.2012      1.636 30
## 4  0.1949      1.632 30
## 5  0.4616      1.523 30
## 6  0.5156      1.522 30
## 7  0.2002      1.634 30
## 8  0.1975      1.634 30
## 9  0.3785      0.886 30
## 10 0.0682      0.000 30
## 11 0.5481      0.985 30
## 12 0.0787      0.000 30
## 13 0.1267      0.241 30
## 14 0.0739      0.123 30
## 15 0.1175      0.348 30
## 16 0.1037      0.269 30
## 17 0.1911      0.309 30
## 18 0.0799      0.119 30
## 19 0.1177      0.355 30
## 20 0.0999      0.273 30
```

Interpretation:

- In well-specified cells, the estimated means should lie near the truth.
- In misspecified cells, the estimates may drift or become more variable.

MAE summary

Mean absolute error is only computed when a meaningful truth exists.

```
mae_df <- est_long[!is.na(est_long$truth), ]

mae_tab <- aggregate(
  abs_error ~ gen_family + fit_family + cell_label + method + parameter,
  data = mae_df,
  FUN = function(x) c(
    mean = mean(x),
    median = stats::median(x),
    sd = sd(x),
    n = length(x)
  )
)

mae_summary <- data.frame(
  gen_family = mae_tab$gen_family,
  fit_family = mae_tab$fit_family,
  cell_label = mae_tab$cell_label,
  method = mae_tab$method,
  parameter = mae_tab$parameter,
  mae_mean = mae_tab$abs_error[, "mean"],
  mae_median = mae_tab$abs_error[, "median"],
```

```

    mae_sd      = mae_tab$abs_error[, "sd"],
    n           = mae_tab$abs_error[, "n"]
)

print(mae_summary, digits = 3)

```

##	gen_family	fit_family	cell_label	method	parameter	mae_mean
## 1	negbin	negbin	negbin → negbin	GHK	Intercept	0.4047
## 2	negbin	poisson	negbin → poisson	GHK	Intercept	0.4447
## 3	poisson	negbin	poisson → negbin	GHK	Intercept	0.1611
## 4	poisson	poisson	poisson → poisson	GHK	Intercept	0.1575
## 5	negbin	negbin	negbin → negbin	GQT	Intercept	0.3963
## 6	negbin	poisson	negbin → poisson	GQT	Intercept	0.4335
## 7	poisson	negbin	poisson → negbin	GQT	Intercept	0.1613
## 8	poisson	poisson	poisson → poisson	GQT	Intercept	0.1599
## 9	negbin	negbin	negbin → negbin	GHK	overdispersion	0.3228
## 10	negbin	negbin	negbin → negbin	GQT	overdispersion	0.4551
## 11	negbin	negbin	negbin → negbin	GHK	range	0.1052
## 12	negbin	poisson	negbin → poisson	GHK	range	0.2099
## 13	poisson	negbin	poisson → negbin	GHK	range	0.1053
## 14	poisson	poisson	poisson → poisson	GHK	range	0.0827
## 15	negbin	negbin	negbin → negbin	GQT	range	0.1506
## 16	negbin	poisson	negbin → poisson	GQT	range	0.2075
## 17	poisson	negbin	poisson → negbin	GQT	range	0.1084
## 18	poisson	poisson	poisson → poisson	GQT	range	0.0794
##	mae_median	mae_sd	n			
## 1	0.3942	0.2542	30			
## 2	0.4647	0.2794	30			
## 3	0.1403	0.1189	30			
## 4	0.1488	0.1140	30			
## 5	0.3752	0.2252	30			
## 6	0.4735	0.2678	30			
## 7	0.1562	0.1177	30			
## 8	0.1562	0.1150	30			
## 9	0.3055	0.2093	30			
## 10	0.4935	0.3159	30			
## 11	0.1038	0.0780	30			
## 12	0.1767	0.0739	30			
## 13	0.0873	0.0699	30			
## 14	0.0700	0.0610	30			
## 15	0.1263	0.1229	30			
## 16	0.1812	0.0798	30			
## 17	0.0880	0.0729	30			
## 18	0.0666	0.0596	30			

Interpretation:

- In the well-specified cells, MAE is the main accuracy metric.
- In misspecified cells, MAE is still interpretable for shared parameters like range and intercept-scale mean, but not always for parameters that only exist under one family.

Information criteria summary

AIC/BIC/AICc are recorded by the wrapper. These can be useful descriptively, but they should be interpreted carefully across misspecified families.

```
ic_df <- fit_df[fit_df$converged, ]

ic_tab <- aggregate(
  cbind(loglik, aic, bic, aicc) ~ gen_family + fit_family + cell_label + method,
  data = ic_df,
  FUN = mean
)

print(ic_tab, digits = 3)
```

	gen_family	fit_family	cell_label	method	loglik	aic	bic
## 1	negbin	negbin	negbin → negbin	GHK	-1.21e+02	2.47e+02	2.53e+02
## 2	negbin	poisson	negbin → poisson	GHK	-1.56e+02	3.16e+02	3.20e+02
## 3	poisson	negbin	poisson → negbin	GHK	-9.50e+01	1.96e+02	2.02e+02
## 4	poisson	poisson	poisson → poisson	GHK	-9.52e+01	1.94e+02	1.98e+02
## 5	negbin	negbin	negbin → negbin	GQT	-1.20e+02	2.46e+02	2.52e+02
## 6	negbin	poisson	negbin → poisson	GQT	-3.18e+13	6.35e+13	6.35e+13
## 7	poisson	negbin	poisson → negbin	GQT	-9.48e+01	1.96e+02	2.01e+02
## 8	poisson	poisson	poisson → poisson	GQT	-9.52e+01	1.94e+02	1.98e+02
##	aicc						
## 1	2.48e+02						
## 2	3.16e+02						
## 3	1.97e+02						
## 4	1.95e+02						
## 5	2.47e+02						
## 6	6.35e+13						
## 7	1.96e+02						
## 8	1.95e+02						

Boxplots of runtime

If ggplot2 is available, produce a clean runtime plot. Otherwise use base graphics.

```
if (requireNamespace("ggplot2", quietly = TRUE)) {
  library(ggplot2)

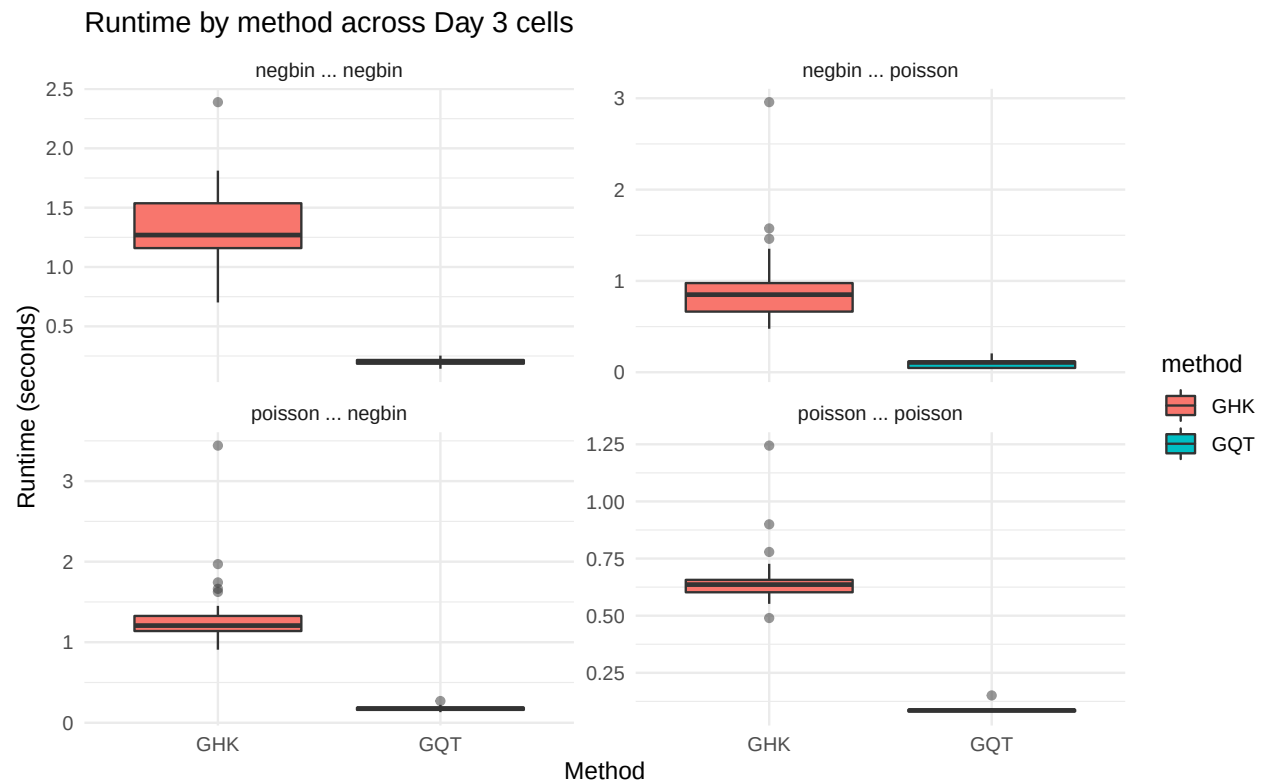
  ggplot(runtime_df, aes(x = method, y = runtime_sec, fill = method)) +
    geom_boxplot(outlier.alpha = 0.5) +
    facet_wrap(~ cell_label, scales = "free_y") +
    labs(
      title = "Runtime by method across Day 3 cells",
      x = "Method",
      y = "Runtime (seconds)"
    ) +
    theme_minimal()
} else {
  boxplot(
    runtime_sec ~ interaction(cell_label, method),
```



```

data = runtime_df,
las = 2,
main = "Runtime by cell and method",
ylab = "Runtime (seconds)",
xlab = ""
)
}

```



Boxplots of parameter estimates

Again, use ggplot2 if available.

```

if (nrow(est_long) > 0) {
  if (requireNamespace("ggplot2", quietly = TRUE)) {
    library(ggplot2)

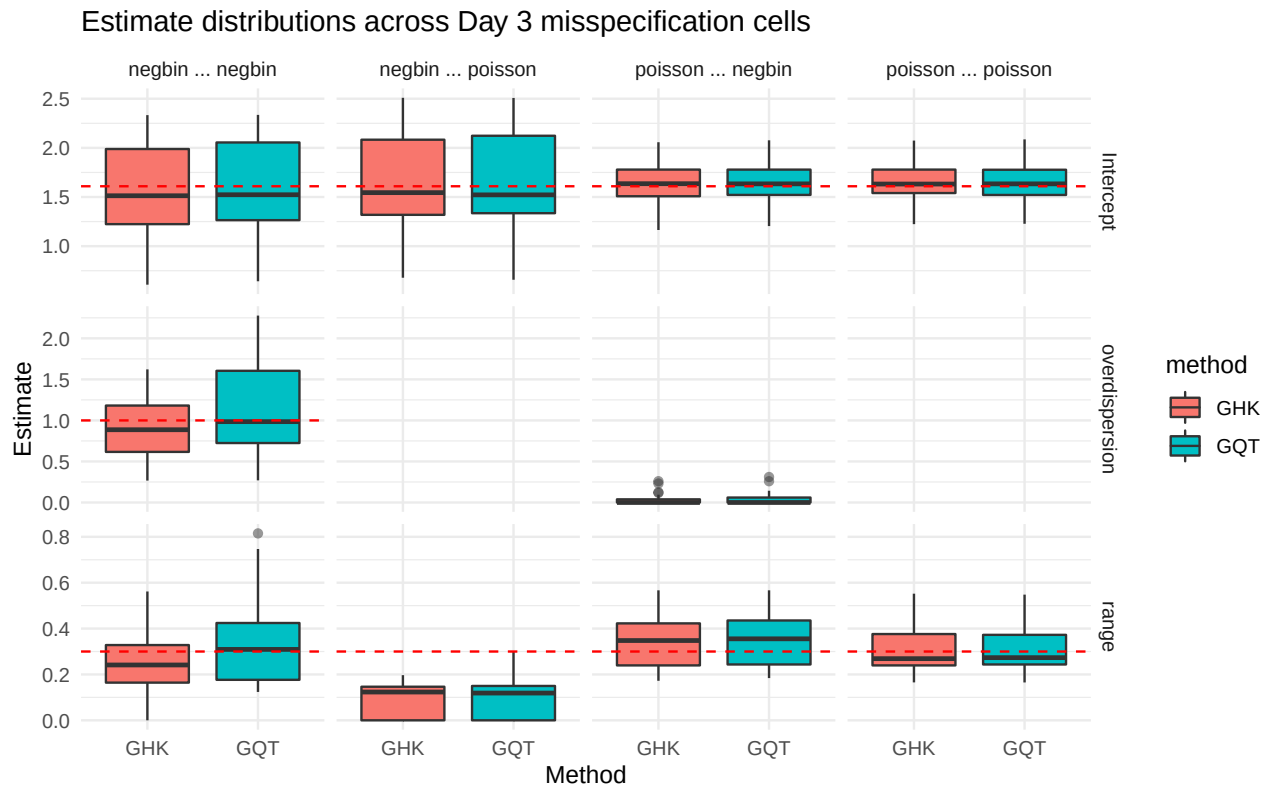
    ggplot(est_long, aes(x = method, y = estimate, fill = method)) +
      geom_boxplot(outlier.alpha = 0.5) +
      facet_grid(parameter ~ cell_label, scales = "free_y") +
      geom_hline(aes(yintercept = truth), colour = "red", linetype = 2, na.rm = TRUE) +
      labs(
        title = "Estimate distributions across Day 3 misspecification cells",
        x = "Method",
        y = "Estimate"
      ) +
      theme_minimal()
  } else {

```

```

par(mfrow = c(1, 1))
boxplot(
  estimate ~ interaction(method, cell_label),
  data = est_long,
  las = 2,
  main = "Estimate distributions",
  ylab = "Estimate",
  xlab = ""
)
}
}

```



Interpretation:

- The dashed red line is the truth where defined.
- In well-specified cells, boxes should be centered closer to the red line.
- In misspecified cells, the distributions may shift or widen.

A simple 2 x 2 Day 3 summary table

This is the most report-like table in the document.

It focuses on:

- convergence,
- mean runtime,
- mean MAE for the parameters where truth exists.

```

# convergence
conv_small <- convergence_summary[, c("cell_label", "method", "conv_rate")]

# runtime
rt_small <- runtime_summary[, c("cell_label", "method", "mean_sec")]

# MAE
mae_small <- mae_summary[, c("cell_label", "method", "parameter", "mae_mean")]

# merge convergence + runtime first
cell_method_tab <- merge(conv_small, rt_small, by = c("cell_label", "method"), all = TRUE)

# reshape MAE wider by parameter
if (nrow(mae_small) > 0) {
  mae_wide <- reshape(
    mae_small,
    idvar = c("cell_label", "method"),
    timevar = "parameter",
    direction = "wide"
  )

  day3_summary <- merge(cell_method_tab, mae_wide, by = c("cell_label", "method"), all = TRUE)
} else {
  day3_summary <- cell_method_tab
}

print(day3_summary, digits = 3)

```

```

##           cell_label method conv_rate mean_sec mae_mean.Intercept
## 1  negbin → negbin   GHK           1   1.3448           0.405
## 2  negbin → negbin   GQT           1   0.2015           0.396
## 3  negbin → poisson  GHK           1   0.9330           0.445
## 4  negbin → poisson  GQT           1   0.0925           0.434
## 5  poisson → negbin  GHK           1   1.3319           0.161
## 6  poisson → negbin  GQT           1   0.1782           0.161
## 7  poisson → poisson GHK           1   0.6581           0.158
## 8  poisson → poisson GQT           1   0.0879           0.160
##  mae_mean.overdispersion mae_mean.range
## 1                      0.323         0.1052
## 2                      0.455         0.1506
## 3                      NA          0.2099
## 4                      NA          0.2075
## 5                      NA          0.1053
## 6                      NA          0.1084
## 7                      NA          0.0827
## 8                      NA          0.0794

```

Interpretation guide

At this stage, the most important comparisons are:

1. Well-specified versus misspecified

Compare:

- Poisson ($\rightarrow$) Poisson versus Poisson ($\rightarrow$) NegBin
- NegBin ($\rightarrow$) NegBin versus NegBin ($\rightarrow$) Poisson

If the misspecified cell has:

- worse convergence,
- slower runtime,
- larger MAE,
- more variable estimates,

then the misspecification study is already producing a meaningful result.

2. GHK versus GQT under misspecification

The Day 1 and Day 2 baseline focused on GHK versus GQT under correct specification.

Day 3 asks a more robust question:

when the fitted marginal family is wrong, does GQT still behave similarly to GHK, or does one method degrade more severely?

The key summaries for that are:

- convergence rates,
- runtime,
- MAE,
- estimate boxplots.

3. Shared-parameter interpretation

Some parameters remain comparable even under misspecification:

- the intercept-scale mean parameter,
- the spatial range.

Other parameters are only meaningful under one family, especially overdispersion.

So the cleanest Day 3 interpretation usually focuses first on:

- range,
- intercept-scale mean,
- convergence,
- runtime.

Final takeaway

The Day 3 misspecification pipeline is now doing exactly what it should:

1. generate controlled Poisson and NegBin spatial count datasets,
2. fit each under both correct and incorrect marginal assumptions,
3. store one fit-result object per cell,
4. summarize how misspecification affects stability, speed, and accuracy.

This document is the analysis layer on top of that pipeline.
